# Wasm in Kernel

## Evolving Threat Detection at Runtime

Wasm I/O - 2026

Mikhail Katychev
Salman Saghafi

MIMIC

# MIMIC

System level threat detection and mitigation

Reconfigure and redeploy during ongoing attacks

Modifying system in production is risky

# Wasm at Mimic

Process system events from kernel to cloud

Wasm-centric product

- Environment customization
- Dynamic update

Component Model

- Help build and maintain an ecosystem
- Virtualization for cross-platform development

# Kernel Programming is Hard!

Low-level system API

- No high-level standard library

Asynchronous environment

- Interrupts
- Context switching
- Callbacks

No protection from Operating System

- Minor bugs cause full system crashes

# Kernel Programming is Hard!

eBPF (extended Berkeley Packet Filter) for Linux

- Change kernel's behavior without restart
- Restricted logic for safety

eBPF for Windows is under development

microsoft/ebpf-for-windows

WA

# Wasm in Windows Kernel

Update code in kernel without restart

Sandboxing for safety guarantees

Policy-driven execution

- Resource limiting
- Fine-grained access control

# Challenges

No standard library (`#[no_std]`)

No executable memory due to kernel security policies

No Just-in-Time (JIT) compilation

# Interpreting Wasm

TinyWasm, Wasmi, and Wasmtime with Pulley

Wasmi is ~10x slower than Wasmtime with JIT

Wasmi is ~20-50% faster than Pulley

# Component Model

Secure and modular ecosystem

Compatibility with user-space ecosystem

Wasmtime was the only viable option

# Kernel Embedding

Run as a minifilter driver

- Intercept and process I/O events
- High-level API

Host functions interact with kernel (no WASI)

Compile to Pulley in user space

# Host Implementation

Enforce `no_std` in Wasmtime

bytecodealliance/wasmtime/pull/11152

IRQL (Interrupt Request Level) aware error handling

Implement host primitives through C API

- Memory mapping via heap allocation
- Structured Exception Handling instead of panic unwinding
- Custom TLS (Thread Local Storage)

| | |
|---|---|
| High Level | 15 |
| IP_LEVEL \| POWER_LEVEL | 14 |
| CLOCK_LEVEL | 13 |
| SYNCH_LEVEL | 12 |
| Device IRQL | 3 to 11 |
| DISPATCH_LEVEL | 2 |
| APC_LEVEL | 1 |
| PASSIVE_LEVEL | 0 |

# Multithreading

Run concurrent Wasm components

Wasmtime `no_std` assumed single threaded host

- Panic on contention

Host provided synchronization primitives

bytecodealliance/wasmtime/pull/11836

# Thread Local Storage

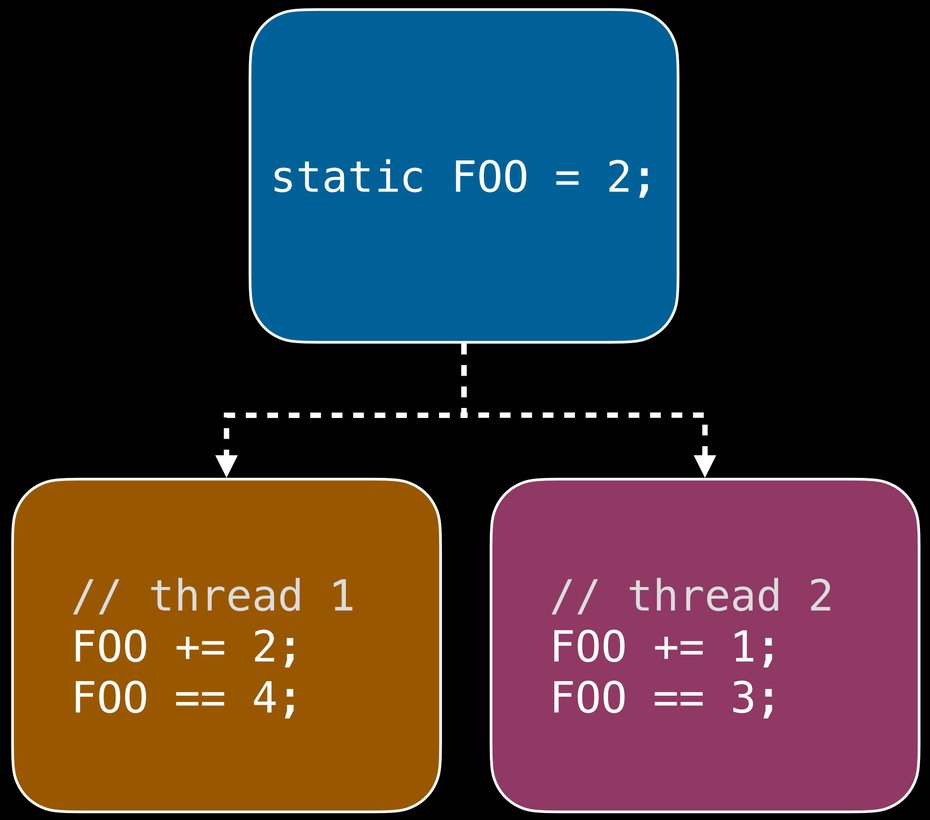
Standard library implements TLS

Custom TLS implementation

- Stack pinned thread scopes and IDs

Thread-scope registry pattern

- Per-thread stack of values



# Wrapping up

Safe kernel development with Wasm

Sandboxing vs. static verification

Be aware of performance cost

# Live Demo